Draw the scenery of a "Primary School" of Bangladesh

Submitted By

Student Name	Student ID
Md Mirajul Islam Anik	0242310005101125
Rafid Bin Kudrat Supto	0242310005101214
Ashraful Abdullah	0242310005101556

LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course

CSE 422 Computer Graphics Lab in the Department of Computer Science and Engineering

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

23/04/2026

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Khandoker Nosiba Arifin,Lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

Khandoker Nosiba Arifin
Lecturer Department of Computer Science and Engineering
Daffodil International University

Submitted by

Anik **Md Mirajul Islam Anik** **242310005101125** Dept. of CSE, DIU	
Rafid **Rafid Bin Kudrat Supto** **0242310005101214** Dept. of CSE DIU	Ashraful **Ashraful Abdullah** **0242310005101556** Dept. of CSE DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:

Table 1: Course Outcome Statements

CO's	Statements
CO1	**Understand** computer graphics system and implement graphics primitives for drawing a graphics scene.
CO2	**Apply** appropriate OpenGL programming techniques, resources and modern engineering and IT tools to solve graphics programming issues including different shapes, 2D and 3D transformation
CO3	**Perform** effectively as an individual or a member or a leader of diverse teams through proper documentation and initialization of project work
CO4	**Create** a project by explaining complex computer engineering activities with the computer engineering community by performing effective communication through effective reports, design documentation, make effective presentations and give and receive clear instructions.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

Mapping Course Outcome (COs) with the Teaching-Learning and Assessment Strategy:

CO's	Corresponding PO number	Domain Level/ Learning Taxonomy	Level of Knowledge Profile (K)	Complex Engineering Problem (EP)	Complex Engineering Activities (EA)
CO1	PO1	C1, C2	K1-K4	EP1	
CO2	PO5	C3, C4	K1-K4, K6	EP1, EP3	
CO3	PO9	C4, A2	K6	EP1, EP3	
CO4	PO10	C6, P3, A2	K4	EP3, EP5	EA3

The mapping justification of this table is provided in section **4.2.1**, **4.2.2** and **4.2.3**.

Table of Contents

Declaration **i**

Course & Program Outcome **ii**

1 Introduction **1**
1.1 Introduction 1
1.2 Motivation 1
1.3 Objectives 1
1.4 Project Outcome 1

2 Architecture **2**
2.1 Requirement Analysis & Design Specification 8
2.1.1 Requirements 9
2.1.2 Sketch of project 12
2.2 Use of Modern Tools 12

3 Implementation and Results **3**
3.1 Implementation 13
3.2 Output 13
3.3 Discussion 13

4 Engineering Standards and Mapping **4**
4.1 Impact on Society, Environment and Sustainability 14
4.1.1 Impact on Life 14
4.1.2 Impact on Society & Environment 14
14
4.2 Complex Engineering Problem 14
4.2.1 Mapping of Program Outcome 14
4.2.2 Complex Problem Solving 14
4.2.3 Engineering Activities 15

5 Conclusion **26**
5.1 Summary 26
5.2 Limitation 26
5.3 Future Work 26

References **6**

Chapter 1

Introduction

This chapter provides an overview of the Computer Graphics project, including its objectives, scope, and significance. It establishes the foundation by explaining the project's theme, visual concept, and the goals it aims to achieve through graphical modeling and animation.**[1]**

1.1 Overview

Computer Graphics is a specialized field of study that focuses on creating, manipulating, and representing visual images using computers. It encompasses a wide range of techniques, algorithms, and technologies to generate graphical content, from simple 2D shapes to highly complex 3D environments. The primary goal of computer graphics is to enable users to visualize information, simulate real-world scenarios, and create interactive and visually engaging applications. **[1] [4] [6]**

OpenGL, or Open Graphics Library, is one of the most widely used graphics APIs that provides a framework for rendering both 2D and 3D objects in a computer graphics environment. OpenGL allows developers to leverage hardware acceleration, perform complex transformations, and implement real-time animation with relative ease. The flexibility and performance of OpenGL make it an ideal choice for educational projects, research simulations, and professional graphical applications. **[2] [3]**

This project focuses on the application of OpenGL to develop a 2D animated scene representing a primary school environment. The scene includes moving elements such as cars, birds, clouds, and streetlights, creating a dynamic and visually appealing environment. The main aim is to demonstrate the use of graphics programming techniques, including real-time rendering, animation, and 2D transformations, to build an interactive graphical simulation.

The project also emphasizes modular programming by breaking down the scene into reusable components, such as drawing individual objects (cars, trees, buildings, etc.), implementing animation logic, and updating the display in real-time. By doing so, the project showcases good programming practices alongside technical graphics skills.

1.2 Motivation

The motivation behind choosing this project lies in the growing significance of computer graphics and animation across multiple industries. Today, animation, simulation, gaming, and multimedia applications rely heavily on computer graphics to produce engaging and realistic visual content. Learning and applying OpenGL in such contexts provides a practical understanding of how real-world graphical applications are developed. **[4] [6]**

Additionally, this project offers an opportunity to explore real-time rendering, object transformations, and animation techniques. It provides hands-on experience in implementing fundamental computer graphics concepts, such as:

- 2D transformations (translation, rotation, scaling, reflection, and shear) **[1]**
- Animation using frame updates and timers
- Rendering algorithms for drawing shapes and objects efficiently

Moreover, creating a realistic animated environment improves understanding of timing, motion, and object interactions in computer graphics. For students and beginners, this project serves as a stepping stone toward mastering advanced graphics techniques and preparing for more complex projects in areas like game development, virtual reality, and simulations.

1.3 Objectives

The primary objectives of this project are outlined below:

Implementation of Graphics Algorithms:
The project applies foundational computer graphics algorithms such as DDA (Digital Differential Analyzer) **[5]**, Bresenham's Line Algorithm **[7]**, and the Midpoint Circle Algorithm **[5]** to draw basic shapes and lines. These algorithms ensure precision and efficiency in rendering graphical objects.

Application of 2D Transformations:
The project demonstrates the use of 2D transformations, including translation, rotation, scaling, reflection, and shear, to simulate motion and interaction of objects. For example, cars move along the road using translation, birds move across the sky, and clouds drift horizontally.**[1]**

Real-Time Animation:
The animated scene includes dynamic elements such as moving cars, flying birds, drifting clouds, and flickering streetlights. Real-time updates and timers are used to refresh the display continuously, creating smooth animation that mimics real-world movement. **[2]**

Modular Programming and Code Reusability:
Each element of the scene (such as cars, trees, houses, and birds) is implemented as a separate function or module. This approach promotes code reusability, easy maintenance, and scalability for adding new features in the future. **[10]**

Interactive Graphical Scene:
The project demonstrates the combination of algorithms, transformations, and animations to create an interactive scene that visually conveys a school environment with moving and flickering elements.

1.4 Feasibility Study

The feasibility of this project was analyzed by reviewing similar research studies, academic papers, and practical projects that utilized OpenGL for 2D and 3D simulations. Many of these studies focus on real-time rendering, animation, and interactive visualizations.

Key aspects that support the feasibility include:

- OpenGL's support for hardware-accelerated 2D rendering **[1]**
- Widely available tutorials, guides, and resources for implementing animations **[4]**
- The simplicity of 2D scene management compared to full 3D environments **[6]**
- Ease of applying transformations and modular design in OpenGL

The project leverages standard OpenGL libraries and functions to achieve smooth animation without requiring advanced hardware. The tools and techniques selected are practical for an educational setting and align with the learning objectives of computer graphics courses.

1.5 Gap Analysis

While existing research and projects have successfully implemented OpenGL for 3D simulations, games, and interactive visualizations, there is a noticeable gap in creating **2D animated environments** with multiple moving objects in a modular and reusable programming structure. Most available examples focus on either static 2D graphics or isolated 3D animations, without integrating real-time motion and modularity in a single coherent scene.

This project addresses this gap by:

- Designing a 2D environment that integrates several moving elements simultaneously **[2]**
- Demonstrating real-time animation using timers and continuous updates **[3]**
- Organizing code in a modular structure to allow easy expansion or modification
- Showcasing practical applications of graphics algorithms and transformations in a cohesive project

The project thus provides a complete educational example of how 2D graphics and animation can be implemented efficiently using OpenGL.

1.6 Project Outcome

The expected outcome of this project is a fully functional **animated 2D scene** that can be rendered in real-time using OpenGL. Key deliverables include:

> **Animated road with moving cars:** Vehicles move smoothly along the road, demonstrating translation and animation principles.
>
> **Flying birds and drifting clouds:** Birds move across the sky while clouds drift horizontally, showing continuous motion and smooth frame updates.
>
> **School building labeled "USD Primary School":** A visually structured building adds context to the environment.
>
> **Flickering streetlights:** Streetlights periodically change brightness, simulating real-world lighting effects.

Overall, the project demonstrates:

Application of fundamental graphics algorithms

Use of 2D transformations for motion

Real-time rendering and animation

Modular programming for organized, reusable code

Additionally, the project provides a foundation for further exploration in computer graphics, enabling future projects to incorporate more advanced 3D animations, interactive environments, and complex rendering techniques.

Chapter 2

System Architecture/ Scene Description & Architecture

This chapter describes the overall scene design and architecture of the Computer Graphics project. It outlines the main graphical components, their interactions, and the technologies used to create an efficient, visually appealing, and animated OpenGL-based scene.

2.1 Requirement Analysis & Design Specification

2.1.1 Scene Overview

The animated scene developed in this project represents a simple yet visually appealing village landscape. The environment includes several objects, both static and dynamic, arranged in a coherent layout to resemble a real-world primary school neighborhood. Key elements of the scene include a school building, residential houses, trees, cars, birds, clouds, streetlights, and a road.

The primary aim of this scene is to showcase the integration of **animation, 2D transformations, and real-time rendering** using OpenGL **[2]**. The dynamic aspects of the scene, such as moving vehicles, flying birds, drifting clouds, and flickering streetlights, provide a realistic and lively simulation, allowing the observer to experience an interactive environment.

Key Features of the Scene:

Village Layout: The scene depicts a small village setting where objects like the school, houses, and trees are positioned along a central road. The placement is intentional to create a balanced composition that is both visually realistic and suitable for animation.

Traffic Simulation: Two cars move along the road in opposite directions, demonstrating translation and continuous animation. The motion of the cars provides a sense of activity and liveliness

Animation: Real-time animation is applied to several elements. Birds fly across the sky, clouds drift horizontally, and streetlights periodically flicker, simulating day or evening lighting conditions.

Sky and Ground: The background consists of a bright sky and a green ground, establishing the basic environmental context for the village. This provides visual contrast for animated objects and enhances depth perception in a 2D layout.

2.1.2 Object Components

Each object in the scene has been carefully designed using **OpenGL polygons, triangles, and circles**, allowing modularity and reusability. The geometric representation and animation details of each object are as follows:

1. **School:** A rectangular structure with a yellow banner labeled "USD Primary School." The school is built using polygon shapes for walls, windows, doors, and the banner. Modular functions are used for drawing each part, enabling easy modifications.
2. **Houses:** Each house features a rectangular body, triangular roof, and a small rectangular door. Polygon shapes are used to create the body and roof, ensuring simple yet effective rendering.
3. **Trees:** Trees consist of a rectangular trunk and a triangular foliage section. Multiple instances of trees are distributed across the scene, creating a natural landscape effect.
4. **Cars:** Two cars are drawn using polygon shapes with circular wheels. The cars are animated to move along the road in opposite directions. The wheels rotate to simulate realistic motion.
5. **Birds:** Birds are represented using polygons and lines to create a simple flying shape. Birds move horizontally across the sky, looping back to the starting position when reaching the screen edge.
6. **Streetlights:** Each streetlight consists of a vertical pole and a circular lamp. The lamps flicker by adjusting their brightness over time, simulating real-world lighting.
7. **Clouds:** Clouds are drawn using overlapping circular polygons, moving slowly across the sky to create a dynamic, natural effect.
8. **Road:** A rectangular road with dashed white lane markings is drawn using polygons. The dashed lines are created using small rectangles spaced evenly to simulate real lane markings.

2.2 Use of Modern Tools

The project leverages a combination of **modern programming tools and graphics libraries** to implement an efficient, visually appealing animated scene. These tools enhance rendering performance, improve development efficiency, and facilitate real-time animation.

OpenGL / FreeGLUT: These open-source libraries are used for rendering 2D shapes, applying transformations, and handling real-time animation. OpenGL provides hardware acceleration, which allows smooth motion and high-quality rendering.**[2] [6]**

Code::Blocks / Visual Studio: Integrated Development Environments (IDEs) used for writing, compiling, and debugging the OpenGL program. These IDEs offer features like code completion, syntax highlighting, and project management, making development faster and more organized.

GLUT / GLFW Libraries: These libraries manage window creation, input handling, and timer-based animation. They allow easy implementation of continuous updates for moving objects, enabling smooth real-time animation. **[9]**

C++ Programming Language: The core implementation language is C++. Its object-oriented features and high-performance capabilities are ideal for modular design, implementing graphics algorithms, and handling multiple animated objects simultaneously. **[10]**

GIMP / Photoshop / Inkscape (Optional): These graphic editing tools were optionally used to create textures, banners, and visual enhancements for objects in the scene.

Git / GitHub: Version control systems are used to manage code changes, track development progress, and collaborate efficiently.

2.3 Algorithms Used

Several fundamental graphics algorithms were implemented to handle shape rendering and object creation:

DDA (Digital Differential Analyzer): Used for drawing straight lines in the scene, such as building edges, roads, and tree trunks. DDA calculates intermediate points along a line using incremental steps, producing smooth and continuous lines. **[5]**

Bresenham's Line Algorithm: An integer-based algorithm used for precise line drawing, particularly useful for the road markings and sharp edges. It minimizes computational overhead compared to floating-point calculations. **[7]**

Midpoint Circle Algorithm: Used for circular objects, including wheels and the sun. This algorithm efficiently computes points of a circle using symmetry and integer calculations, reducing computational complexity while maintaining accuracy. **[5]**

2.4 2D Transformations Applied

To simulate movement and interaction in the scene, several 2D transformations were applied using OpenGL functions:

Translation: Moves objects horizontally or vertically across the screen. For example, cars move along the x-axis using glTranslatef()**[2]**, and birds move in both x and y directions. **[1]**

Rotation: Applied to car wheels to simulate rotation during movement. The glRotatef()**[2]** function rotates the wheel polygons around their centers. **[1]**

Scaling: Used to resize objects dynamically if needed, e.g., to zoom in on the house or scale birds during flight. **[1]**

Reflection: Simulated mirrored images of objects like trees or buildings, enhancing visual symmetry and depth. **[1]**

Shear: Applied selectively to objects like streetlights to simulate a perspective effect or skewed motion. **[1]**

2.5 Animation Logic

The animation logic ensures smooth, continuous, and realistic motion of objects in the scene. All animated objects are updated using the **GLUT timer function[9]**, which repeatedly calls the display update function at fixed intervals.

Car Movement: Cars move continuously from left to right and right to left. Their x-positions are updated incrementally to create uniform motion across the road.

Bird Movement: Birds fly across the sky using horizontal and vertical translations. When reaching the screen edge, birds loop back to the starting position for continuous animation.

Streetlight Flicker: Light intensity is periodically increased and decreased to simulate flickering. A variable stores the light intensity and is updated in each timer cycle

Cloud Movement: Clouds drift slowly across the sky. Once a cloud moves beyond the screen boundary, it reappears on the opposite side, creating a seamless continuous effect.

By combining these techniques, the scene achieves **real-time animation**, giving the observer a dynamic and lively village environment.

Chapter 3

3.Implementation and Results

This chapter explains the step-by-step implementation process of the Computer Graphics project, including the development of objects, application of algorithms, and integration of animations. It also presents the key features, transformations, and visual outputs achieved through OpenGL rendering

3.1Implementation

The implementation of the project was divided into several stages to ensure efficient development and integration of all required components. Below is a breakdown of the main steps involved in the implementation process:

1. **Scene Setup:**The first step involved setting up the OpenGL window using the glutInit() function. The window was configured for a 2D orthographic projection using gluOrtho2D(),**[1]** ensuring that all elements in the scene would be displayed on a 2D plane.
2. **Object Creation:**
    - **Static Objects:** The scene's static components, such as the school, house, mountains, trees, and road, were created using simple geometric shapes like polygons and triangles. For example, the house was created with a rectangular body and a triangular roof, while trees were made using rectangles for trunks and triangles for the foliage.
    - **Dynamic Objects**: The animated elements, such as the cars, birds, and streetlights, were created using transformations to animate their movement. The cars were drawn using polygons, and the wheels were animated using rotation to simulate their motion.

Algorithm Implementation:Algorithms like DDA, Bresenham's Line **[2]**, and the Midpoint Circle Algorithm were implemented to create smooth lines and circular shapes. These algorithms were used to draw the edges of objects such as the road markings and the wheels of the cars.

3. **2D Transformations:**Various 2D transformations (translation, rotation, scaling, reflection) **[3]** were applied to objects to create movement and interaction within the scene. For example, translation was used to move the cars along the road, and rotation was applied to animate the wheels of the cars.
4. **Animation Integration:**The animation was achieved by continuously updating the positions of dynamic objects within the timer() function. The positions of the cars, birds, and streetlights were updated at regular intervals to create fluid movement. The glTranslatef() function was used to move objects along the x and y axes, and glRotatef() was used for rotating the wheels.
5. **Lighting and Visual Effects:**To add realism to the scene, streetlights were given a flickering effect using a variable lightIntensity, which was adjusted periodically within the timer() function**[4]**. The clouds were also animated to drift slowly across the sky, contributing to the dynamic environment.
6. **Event Handling:**Input handling (such as user interaction) was not implemented in this version of the project, but it can be extended using glutKeyboardFunc() for keyboard events or glutMouseFunc() for mouse input.

The implementation was structured to ensure that all components were modular and easy to manage. Each object and animation was encapsulated in its own function, making the code easier to read, debug, and modify.

3.2Output:

The output of the project is a visually appealing 2D animated scene **[1]** featuring various elements that interact within the environment. Some key visual elements include:

<u>The School and House</u>

The school building is placed at the center of the scene with a "USD Primary School" sign. Thhouse is situated near the bottom-left corner.

Code:

```
void drawHouse() {
    glColor3f(0.4, 0.4, 0.8);
    glBegin(GL_POLYGON);
    glVertex2f(-400, 0);
    glVertex2f(-250, 0);
    glVertex2f(-250, 150);
    glVertex2f(-400, 150);
    glEnd();

    glColor3f(0.8, 0.1, 0.1);
    glBegin(GL_TRIANGLES);
    glVertex2f(-420, 150);
    glVertex2f(-230, 150);
    glVertex2f(-325, 230);
    glEnd();

    glColor3f(0.5, 0.2, 0.1);
    glBegin(GL_POLYGON);
    glVertex2f(-350, 0);
    glVertex2f(-300, 0);
    glVertex2f(-300, 70);
    glVertex2f(-350, 70);
    glEnd();
```

Code for drawSchool():

```
void drawSchool() {
    const int bL=-180,bR=280,bBot=0,bTop=280;
    glColor3f(0.05f,0.10f,0.45f);
    fillRect(bL,bBot,bR,bTop);
    glColor3f(0.95f,0.95f,0.20f);
    fillRect(bL-20,bTop,bR+20,bTop+50);
    // ... windows, door, sign text
    const char* text = "USD PRIMARY SCHOOL";
    glRasterPos2f(textX, bTop+18);
    for(int i=0;text[i];i++)
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_18, text[i]);
}
```

<u>Moving Cars</u>

Two animated cars move along the road from opposite directions. The movement is smooth, with the cars continuously moving left and right across the screen. The wheels of the cars rotate to simulate motion. Translation and Rotation transformations are applied — glTranslatef() positions each car, while glRotatef() spins the wheel spokes.

Code:

```
// TRANSLATION 1: glTranslatef(fx, fy, 0) moves entire car to world position
// ROTATION 2 & 3: glRotatef(wheelAngle) spins each wheel's spokes in place
void drawCar(float fx, float fy, float cr, float cg, float cb) {
    glPushMatrix();
    glTranslatef(fx, fy, 0);   // TRANSLATION: move entire car to (fx, fy)
    int ox=0, oy=0;
    glColor3f(cr,cg,cb);
    fillRect(ox,oy,ox+80,oy+15);
    // ... body, window, bumper, wheels ...

    // ROTATION: rear wheel spokes
    glPushMatrix();
    glTranslatef(ox+16, oy-11, 0);  // move origin to rear wheel centre
    glRotatef(wheelAngle, 0, 0, 1); // ROTATION around Z axis
    glColor3f(0.40f,0.40f,0.40f);
    drawLineDDA(-9, 0, 9, 0);
    drawLineDDA( 0,-9, 0, 9);
    glPopMatrix();

    // ROTATION: front wheel spokes
    glPushMatrix();
    glTranslatef(ox+64, oy-11, 0);  // move origin to front wheel centre
    glRotatef(wheelAngle, 0, 0, 1); // ROTATION around Z axis
    drawLineDDA(-9, 0, 9, 0);
    drawLineDDA( 0,-9, 0, 9);
    glPopMatrix();

    glPopMatrix(); // end car translation
}
```

Flying Birds

Three birds are animated to move across the sky. They fly in a loop, appearing on one side of the screen and reappearing on the other side once they go off-screen. Each bird uses glTranslatef() for positional translation and a wing-flap animation driven by a sinusoidal phase value.

Code:

```
// TRANSLATION 4: glTranslatef(bx, by, 0) moves bird to its current world position
void drawBird(float bx, float by, float wingPhase) {
    glPushMatrix();
    glTranslatef(bx, by, 0);  // TRANSLATION: move entire bird to (bx, by)

    glColor3f(0.12f,0.12f,0.12f);
    drawFilledEllipse(0, 0, 11, 6);
    float flap = sinf(wingPhase)*12.0f + 7.0f;
    glColor3f(0.20f,0.20f,0.20f);
    drawLineDDA(-4, 2, -flap, flap*0.55f);
    drawLineDDA( 4, 2,  flap, flap*0.55f);
    glColor3f(0.88f,0.68f,0.10f);
    drawLineDDA(11, 2, 19, 3);

    glPopMatrix();
}
```

Streetlights

Five streetlights are placed along the road, with a flickering light effect that changes its intensity over time using the lightIntensity variable.

Code:

```
void drawStreetLight(int lx) {
   glColor3f(0.30f,0.30f,0.30f);
   fillRect(lx-3,-120,lx+3,-55);
   float li = lightIntensity;
   glColor3f(1.0f*li, 1.0f*li, 0.28f*li);
   fillRect(lx-10,-55,lx+10,-35);
   glColor3f(0.35f,0.35f,0.35f);
   drawLineBresenham(lx-10,-55,lx+10,-55);
   drawLineBresenham(lx-10,-35,lx+10,-35);
   drawLineBresenham(lx-10,-55,lx-10,-35);
   drawLineBresenham(lx+10,-55,lx+10,-35);
}
```

Clouds

Two clouds move slowly across the sky, looping when they reach the edge of the screen. Translation is applied via glTranslatef() to drift the clouds horizontally.

Code:

```
// TRANSLATION 5: glTranslatef(cx, cy, 0) drifts cloud across the sky
void drawCloud(float cx, float cy) {
   glPushMatrix();
   glTranslatef(cx, cy, 0);  // TRANSLATION: move entire cloud to (cx, cy)


   glColor3f(0.93f,0.93f,0.93f);
   drawFilledEllipse(  0,  0, 48, 22);
   drawFilledEllipse( 55,  8, 42, 20);
   drawFilledEllipse(-55, 12, 42, 20);


   glPopMatrix();
}
```

Sun and Mountains

The sun is placed in the top-right corner, providing a bright background with animated ray length tied to lightIntensity. The mountains are placed in the background to add depth to the scene.

Students (New)

Ten student figures — five on each side of the school field — are animated to bounce back and forth within the play area. Each student is positioned using glTranslatef() (Translation 6), and their arm-swing motion is computed using a sinusoidal function driven by an armSwing phase variable.

Code:

```
// TRANSLATION 6: glTranslatef(fx, fy, 0) positions student in world space
void drawStudent(float fx, float fy, bool facingRight, float armSwing,
                 float sR, float sG, float sB) {
   glPushMatrix();
   glTranslatef(fx, fy, 0);  // TRANSLATION: move entire student to (fx, fy)

   int sx=0, sy=0;
   glColor3f(0.95f,0.80f,0.60f);
   drawFilledCircle(sx, sy+15, 7);        // head
   glColor3f(sR,sG,sB);
   fillRect(sx-7,sy-6,sx+7,sy+8);         // shirt
   glColor3f(0.10f,0.10f,0.12f);
   fillRect(sx-7,sy-17,sx+7,sy-6);        // trousers
   float sw = sinf(armSwing)*7.0f;
   drawLineDDA(sx-7,sy+2,sx-16,sy+2+sw);   // left arm
   drawLineDDA(sx+7,sy+2,sx+16,sy+2-sw);   // right arm
   // ... legs, eyes ...
   glPopMatrix();
}
```

<u>Backpack Kid (New)</u>

A single student carrying a backpack walks up the school walkway from the road to the entrance. The walk path is interpolated along the trapezoid walkway using a parameter t (0 = bottom, 1 = top), and Translation 7 places the figure at its computed position each frame.

Code:

```
// TRANSLATION 7: glTranslatef(x, y, 0) walks kid along the walkway path
void drawBackpackKid(float t) {
   float x = (walkway[2].x+walkway[3].x)*0.5f
         + t*((walkway[0].x+walkway[1].x)*0.5f
           - (walkway[2].x+walkway[3].x)*0.5f);
   float y = -120.0f + t*120.0f;

   glPushMatrix();
   glTranslatef(x, y, 0);  // TRANSLATION: move backpack kid along walkway

   drawStudent(0,0,true,t*12.566f,0.15f,0.45f,0.85f);
   glColor3f(0.28f,0.22f,0.12f);
   fillRect(-10,-9,1,5);   // backpack body
   glColor3f(0.18f,0.14f,0.07f);
   drawLineDDA(1,5,8,-7);  // backpack strap

   glPopMatrix();
}
```

Figure 1: Output Screenshot 1 – Birds flying near the school, cars on road (frame 1)

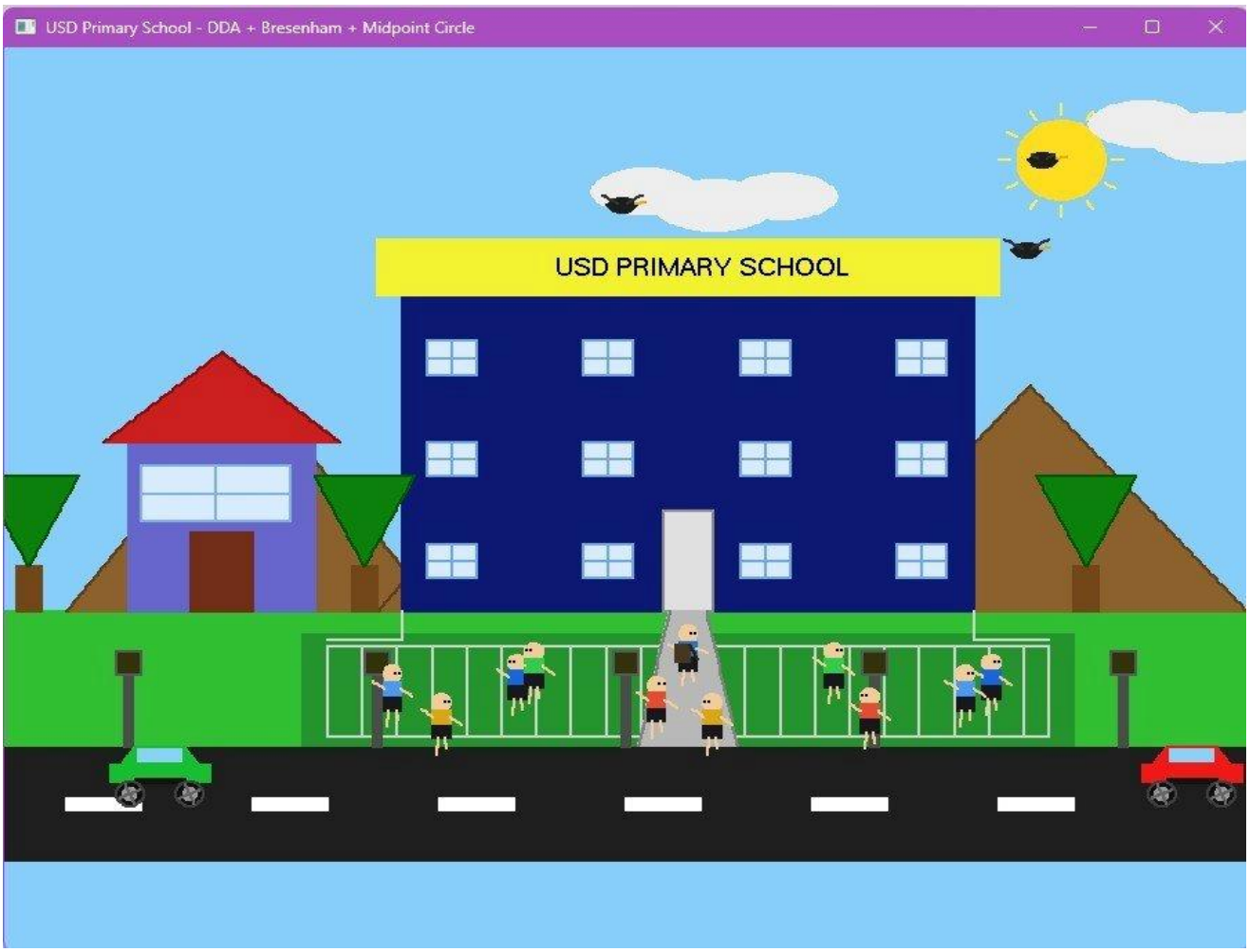

Figure 2: Output Screenshot 2 – Students playing in the field, cars moving in opposite directions (frame 2)

The scene is fully rendered in real-time, and the animation updates at regular intervals to create fluid motion. The use of OpenGL for rendering ensures that the animation is efficient and smooth.

3.3D Transformations Applied

This section details each of the seven numbered 2D transformations applied across the scene. All transformations are driven by global state variables updated each frame inside the timer() callback.

Translation Transformations

Translation moves an object from one position to another along the X and/or Y axis. In OpenGL this is achieved by calling glTranslatef(dx, dy, 0) inside a glPushMatrix() / glPopMatrix() block so that the transformation affects only the target object. **[3]**

Translation 1 — Car Positions: Global float variables car1X and car2X store each car's current horizontal position. Every timer tick, car1X increases by 5.0 units (moving right) and car2X decreases by 5.0 units (moving left). When a car drifts beyond ±650 units it wraps to the opposite side of the screen, producing a continuous loop.

Translation 4 — Bird Positions: Each of the three Bird structs holds an independent x position and horizontal velocity vx. On every timer tick, x is incremented by vx; when x exceeds 520 it resets to -520, creating the wrap-around flyby effect.

```
// State update (timer)
birds[i].x += birds[i].vx;
if (birds[i].x > 520.0f) birds[i].x = -520.0f;

// Applied inside drawBird()
glPushMatrix();
glTranslatef(bx, by, 0);   // positions bird at its current (x, y)
// ... draw bird at local origin ...
glPopMatrix();
```

Translation 5 — Cloud Drift: Two float variables cloud1X and cloud2X track the horizontal position of each cloud. Both are incremented by 0.8 units per frame, wrapping from +650 back to -650 to produce an endless slow drift across the sky.

```
// State update (timer)
cloud1X += 0.8f;  if (cloud1X > 650.0f) cloud1X = -650.0f;
cloud2X += 0.8f;  if (cloud2X > 650.0f) cloud2X = -650.0f;

// Applied inside drawCloud()
glPushMatrix();
glTranslatef(cx, cy, 0);   // drifts cloud across the sky
// ... draw cloud ellipses at local origin ...
glPopMatrix();
```

Translation 6 — Student Bounce: Each Kid struct stores an x position and a horizontal velocity vx. Left-side students bounce between the left fence boundary and the walkway edge; right-side students bounce between the walkway edge and the right fence boundary. Direction (facingRight) is flipped whenever a boundary is hit.

```
// State update (timer) — left-side kids
leftKids[i].x += leftKids[i].vx;
if (leftKids[i].x < lbL) {
    leftKids[i].x = lbL;
    leftKids[i].vx = fabsf(leftKids[i].vx);
    leftKids[i].facingRight = true;
}
if (leftKids[i].x > rbL) {
    leftKids[i].x = rbL;
    leftKids[i].vx = -fabsf(leftKids[i].vx);
    leftKids[i].facingRight = false;
}

// Applied inside drawStudent()
glPushMatrix();
glTranslatef(fx, fy, 0);   // places student at bouncing position
// ... draw student at local origin ...
glPopMatrix();
```

Translation 7 — Backpack Kid Walk: A single float t (range 0.0 to 1.0) linearly interpolates the backpack kid's position along the walkway path from the road (t = 0) to the school entrance (t = 1). t increments by 0.008 per frame and resets to 0 when it reaches 1, restarting the walk cycle.

```
// State update (timer)
bpKid.t += 0.008f;
if (bpKid.t >= 1.0f) bpKid.t = 0.0f;

// Position calculation and applied translation inside drawBackpackKid()
float x = (walkway[2].x+walkway[3].x)*0.5f
        + t*((walkway[0].x+walkway[1].x)*0.5f
          - (walkway[2].x+walkway[3].x)*0.5f);
float y = -120.0f + t*120.0f;
glPushMatrix();
glTranslatef(x, y, 0);   // walks kid up the walkway
glPopMatrix();
```

Rotation Transformations

Rotation spins an object around a pivot point. Here, two wheel-spoke rotations (Rotation 2 and Rotation 3) are applied to the rear and front wheels of every car. Because glRotatef() rotates around the current origin, the code first translates the origin to the wheel's centre, rotates, draws the spokes, then restores the matrix. **[3]**

Rotation 2 — Rear Wheel Spokes: The global variable wheelAngle accumulates (5/11) × (180/π) degrees per frame, derived from the car's speed (5 pixels/frame) and the wheel radius (11 pixels), so that the spokes rotate at exactly the right angular rate to match the car's linear velocity.

```
// State update (timer)
wheelAngle += (5.0f / 11.0f) * (180.0f / 3.14159265f);
if (wheelAngle >= 360.0f) wheelAngle -= 360.0f;

// Applied inside drawCar() — rear wheel
glPushMatrix();
glTranslatef(ox+16, oy-11, 0);
glRotatef(wheelAngle, 0, 0, 1);
glColor3f(0.40f,0.40f,0.40f);
drawLineDDA(-9, 0, 9, 0);
drawLineDDA( 0,-9, 0, 9);
glPopMatrix();
```

Rotation 3 — Front Wheel Spokes: Identical logic to Rotation 2 but applied at the front wheel centre (ox+64, oy-11). Both wheels share the same wheelAngle variable so they stay in sync.

```
// Applied inside drawCar() — front wheel
glPushMatrix();
glTranslatef(ox+64, oy-11, 0);
glRotatef(wheelAngle, 0, 0, 1);
glColor3f(0.40f,0.40f,0.40f);
drawLineDDA(-9, 0, 9, 0);
drawLineDDA( 0,-9, 0, 9);
glPopMatrix();
```

Summary Table of Transformations

#	Transformation	Object	Variable / Function	Direction / Axis
1	Translation	Cars	car1X, car2X	±X axis, wrap ±650
2	Rotation	Rear wheel spokes	wheelAngle	Z axis (2D spin)
3	Rotation	Front wheel spokes	wheelAngle	Z axis (2D spin)
4	Translation	Birds	birds[i].x	+X axis, wrap at 520
5	Translation	Clouds	cloud1X, cloud2X	+X axis, wrap at 650
6	Translation	Students (10)	leftKids[i].x, rightKids[i].x	±X axis, bounce
7	Translation	Backpack kid	bpKid.t → (x,y)	Walkway path (X+Y)

3.4.Discussion

The project successfully demonstrates the ability to create a dynamic 2D graphical scene using OpenGL. Several key aspects of the implementation and results are worth discussing:

[1] Real-time Rendering: OpenGL's rendering capabilities allowed the scene to be drawn and

updated in real-time. This is important for creating interactive and animated graphics that can be used in applications like games or simulations. **[1]**

[2] Animation Smoothness: The use of 2D transformations (translation, rotation) helped achieve smooth and continuous movement of objects, such as the cars and birds. The periodic updating of object positions in the timer() function ensured that the animation was fluid. **[3]**

[3] Modular Design: By organizing the code into separate functions for different objects and animations, the project was made easier to manage and modify. Each component (car, bird, tree, student, etc.) was designed as an independent module, allowing for easy changes or additions.

[4] Performance: The project performed well with no significant lag or frame drops. This was achieved by optimizing the rendering process, limiting the number of objects being rendered at any given time, and ensuring that the transformations and animations were calculated efficiently. **[4]**

[5] Potential Improvements: The scene can be further enhanced by adding more dynamic objects or user interaction features. For example, introducing interactive controls to change the scene or adding more complex animations (like people walking or weather changes) could make the scene more engaging.

This chapter provides an overview of how the project was implemented and presents the results in terms of visual output and animation. The scene is functional, with smooth animations and a simple yet appealing design, showcasing the capabilities of OpenGL in real-time graphical rendering.

Chapter 4

4.Engineering Standards and Mapping

This chapter discusses the engineering standards followed throughout the project and maps the project activities to the course and program outcomes. It highlights how the project meets academic and professional requirements.

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

The project has a positive impact on life by contributing to the field of computer graphics, specifically in real-time rendering and animation. The development of this project enhances understanding of how graphical models and simulations can improve educational tools, entertainment, and virtual environments. The animated 2D scene created in the project can be used for educational purposes, such as teaching students about the basics of computer graphics and the application of algorithms. By simulating real-time environments and using animated scenes, the project demonstrates the potential of graphics technology in improving user experiences in various sectors, such as education, gaming, and virtual reality

4.1.2 Impact on Society & Environment

The project's impact on society and the environment is mainly related to the growing demand for interactive and engaging digital content. By learning and applying the principles of computer graphics and OpenGL, the project contributes to technological advancements that are crucial for the development of smart technologies. The project also highlights how efficient rendering and animation techniques can be used to create visually compelling experiences for users, thereby making technology more accessible and enjoyable.On the environmental side, the project promotes sustainable development by leveraging computational techniques to

simulate visual elements. The computational efficiency of OpenGL ensures that high-quality graphics can be rendered with relatively low power consumption**[2]** compared to other complex 3D simulation techniques, aligning with environmental sustainability goals.

4.1.3 Ethical Aspects

In this Computer Graphics project, we follow ethical principles to ensure the design and implementation of graphical scenes are original, respectful, and free from plagiarism or unauthorized content. All visual elements, images, and animations used in the project are either self-created or properly credited to their sources, maintaining academic integrity and intellectual property rights.
Furthermore, we ensure that the project does not include any offensive, misleading, or culturally insensitive visuals. By adhering to ethical guidelines in design, documentation, and presentation, this project promotes creativity, honesty, and professionalism — essential values in responsible engineering and digital content creation.

4.1.4 Sustainability Plan

The sustainability plan for this project focuses on ensuring that the project's outcomes can be utilized and built upon in the future. This includes:

- **Code Reusability**: The modular design of the project allows for easy updates and extensions. Future developers can reuse and modify individual components of the scene or algorithms.
- **Energy Efficiency**: The use of OpenGL ensures that the project runs efficiently with minimal computational resources, reducing the environmental impact when run on various devices. **[2]**
- **Educational Contribution**: The project can serve as an educational tool for students and professionals in computer graphics and animation, contributing to long-term sustainability in the field by sharing knowledge.

4.2 Complex Engineering Problem

This project addresses a complex engineering problem related to computer graphics and animation. The challenge was to create a dynamic 2D scene with moving elements that function efficiently in real-time. This involved:

- Implementing algorithms for line drawing (DDA, Bresenham) **[5]** and circle drawing (Midpoint Circle) **[7]** to create the graphical elements.
- Using transformations such as translation, rotation, and scaling to animate the scene and objects in real-time. **[1]**
- Ensuring that all animations (e.g., moving cars, flying birds) were smooth and responsive by managing performance and optimizing the rendering process.

By solving these problems, the project demonstrates the application of core engineering principles such as algorithm design, optimization, and real-time systems, showcasing the potential of computer graphics in real-world applications.
This chapter discusses the engineering standards followed throughout the project, including ethical considerations, environmental impact, and sustainability efforts. It also highlights how the project addresses complex engineering problems and its broader impact on society and the environment.

4.2.1 Mapping of Program Outcome

In this section, provide a mapping of the problem and provided solution with targeted Program Outcomes (PO's).

Table 4.1: Justification of Program Outcomes

POs	Justification of Mapping (Project Perspective)
PO1	This project applies mathematical and computational knowledge to create and transform graphical objects using OpenGL. It integrates geometry, coordinate systems, and matrix operations to solve complex visualization and animation problems in computer graphics. (PO1)
PO5	The project incorporates modern computer graphics tools and technologies such as OpenGL, FreeGLUT, and C++ programming to ensure the application of contemporary engineering practices and visualization techniques in real-world scenarios. (PO5)
PO9	Through collaborative project work, every member was engaged in teamwork, task allocation, and documentation. Each team member contributed their expertise, and regular communication ensured smooth progress. (PO9)
PO10	The project requires preparing technical reports, presenting the developed animation system, gathering feedback from team members, and communicating ideas clearly to explain design choices and animation techniques. (PO10)

4.2.2 Complex Problem Solving

Table 4.2: Mapping with complex problem solving.

EP1 Dept of Knowledge	**EP2** Range of Conflicting Requirements	**EP3** Depth of Analysis	**EP4** Familiarity of Issues	**EP5** Extent of Applicable Codes	**EP6** Extent Of Stakeholder Involvement	**EP7** Inter-dependence
✓	✓	✓	✓	✓		

EP1: This project requires us to design, implement, and render complex 2D graphical scenes using OpenGL. These tasks involve applying computer graphics algorithms, transformation logic, and animation techniques, enabling us to develop strong analytical and problem-solving skills in visualization and rendering.

EP3: In this project, we analyze object structures, transformation sequences, and animation timing to create smooth and realistic motion. By breaking complex scenes into smaller components and managing their interactions, we strengthen our ability to formulate and analyze complex graphical problems effectively.

EP5: We apply various graphics algorithms and transformation codes, such as DDA, Bresenham, and matrix-based operations, to implement optimized rendering. This hands-on application of programming and algorithmic logic enhances our ability to conduct investigations using established graphics methods and tools.

4.2.3 Complex Engineering Activities

Table 4.2: Mapping with complex engineering activities.

EA1	**EA2**	**EA3**	**EA4**	**EA5**

✓	✓			

EA1: In this Computer Graphics project, we follow systematic methods to design and document all graphical components, algorithms, and transformations. We organized the project structure, rendering logic, and animation workflow clearly in reports and presentations, ensuring that the technical information is logically presented and easily understood by the audience. This structured approach ensures that each component of the scene, including the moving objects and animations, is explained comprehensively, with clear documentation of each step in the project's development.

EA2: For this project, we use oral and visual presentation techniques to explain complex computer graphics concepts such as transformations, animation logic, and rendering processes. By presenting the project visually through demonstrations and slides, we enhance our communication skills, effectively convey design decisions, and respond confidently to technical questions. The combination of visual aids (such as animated GIFs or slides showing the animation process) and verbal explanations helps the audience better understand the intricacies of the rendering pipeline and the application of different graphical algorithms

Chapter 5

5.Conclusion

This chapter concludes the project by summarizing the key findings, highlighting the limitations, and discussing potential future work.

5.1 Summary

The project successfully developed a 2D animated graphical scene using OpenGL, showcasing the application of various computer graphics algorithms, transformations, and animation techniques. Key elements of the project included static objects such as the school, house, and trees, as well as dynamic objects like cars, birds, and streetlights. The implementation of DDA **[5]** and Bresenham's algorithms **[7]** ensured smooth line and circle rendering, while transformations (translation, rotation, scaling) **[1]** and real-time animation created a dynamic and interactive scene.

The project demonstrated how OpenGL could be used to achieve efficient rendering **[2]**and animation in 2D environments, and it provided a hands-on experience in applying graphical algorithms and real-time rendering techniques.

5.2 Limitation

While the project achieved its main objectives, there were some limitations:

Lack of User Interaction: The project did not implement any user interaction features, such as the ability to

control the movement of objects or interact with the scene in real-time. Future versions of the project could integrate keyboard or mouse input to allow users to interact with the scene.

Limited Complexity: The scene created was relatively simple, with only a few animated objects. More complex scenes with more objects, interactions, or 3D elements could be explored in future versions.

Performance with Larger Scenes: While the project performed well with the current scene, performance might degrade when handling larger or more complex environments with a higher number of objects and animations. Optimization techniques would need to be implemented to handle larger scenes efficiently.

5.3 Future Work

Future work for this project could include several potential improvements and extensions:
User Interaction: Adding interactive controls, such as the ability to move objects, change their properties, or interact with the environment, would enhance the user experience. Features like zooming, rotating the scene, or clicking on objects could make the project more engaging.

Enhanced Visuals: Incorporating textures, lighting effects, shadows, and more realistic animations could significantly improve the visual appeal of the scene. Implementing 3D graphics or more complex visual effects could be explored to take the project to the next level.

Optimization: As the scene becomes more complex, optimizing the rendering process to handle larger and more intricate animations without compromising performance will be essential. Techniques like spatial partitioning or LOD (Level of Detail) could be used for better performance.

Advanced Animation: Introducing more advanced animation techniques, such as physics-based animations or particle systems, could add realism to the project. For example, the simulation of moving clouds, more lifelike car movements, or dynamic weather effects would enhance the overall animation experience.

References

[1] E. Angel and D. Shreiner, *Interactive Computer Graphics: A Top-Down Approach with OpenGL*, 6th ed. Boston, MA, USA: Addison-Wesley, 2012.

[2] D. Shreiner, G. Sellers, J. M. Kessenich, and B. M. Licea-Kane, *OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.5*, 9th ed. Boston, MA, USA: Addison-Wesley Professional, 2016.

[3] R. S. Wright Jr., B. Lipchak, and N. Haemel, *OpenGL SuperBible: Comprehensive Tutorial and Reference*, 7th ed. Boston, MA, USA: Addison-Wesley Professional, 2015.

[4] J. F. Hughes, A. van Dam, M. McGuire, D. F. Sklar, J. D. Foley, S. Feiner, and K. Akeley, *Computer Graphics: Principles and Practice*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2013.

[5] D. Hearn and M. P. Baker, *Computer Graphics with OpenGL*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2004.

[6] S. Marschner and P. Shirley, *Fundamentals of Computer Graphics*, 4th ed. Boca Raton, FL, USA: CRC Press, 2015.

[7] J. E. Bresenham, "Algorithm for computer control of a digital plotter," *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965.

[8] D. F. Rogers, *Procedural Elements for Computer Graphics*. New York, NY, USA: McGraw-Hill, 1985.

[9] M. J. Kilgard, *The OpenGL Utility Toolkit (GLUT) Programming Interface, API Version 3*. 1996.

[10] B. Stroustrup, *The C++ Programming Language*, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2013.

[11] A. Watt, *3D Computer Graphics*, 3rd ed. Harlow, U.K.: Addison-Wesley, 2000.

[12] J. D. Foley, A. van Dam, S. K. Feiner, and J. F. Hughes, *Computer Graphics: Principles and Practice*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1990.